

DATASTRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	DIVYA SAHITHI MASINENI
ROLL NO	AV.SC.U4CSE25129
YEAR/SEM/SECTION	2ndYEAR/ SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(SL Grade)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

Sno	Program	Page No.	**Remarks**
1.	WEEK 1: A. A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by "launch". B. **T**he banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years. C. An HIR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. D. In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number. E. Write a recursive python program to print the first n terms of the Fibonacci series.		
2.	WEEK 2: A. Write a program to perform linear search on a given input. B. (i) Write a python program to implement binary search with sorted array as an input. (ii) Write a python program to implement binary search without sorted array as an input		
3.	WEEK 3:		

	A.Write a python program to implement bubble sort, where the input elements are taken from the user. B. Write a python program to implement insertion sort, where the input elements are taken from the user. C. Write a python program to implement selection sort, where the input elements are taken from the user.		

LAB-1

1.AIM: A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):
 for i in range(n,-1,-1):
 if i==0:
 print("launch")
 else:
 print(i)
n=int (input ("Enter  countdown number:"))
launch(n)
```

OUTPUT:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab1a.py ==========
Enter countdown number: 7
7
6
5
4
3
2
1
Launch
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error	Use 3 **spaces** for each level of indentation.
2.	Type Error	Convert the input to an integer using n = int (input ("Enter no. of seconds to launch: ")).

2.AIM: The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years.

PROGRAM:

```
def power (p, n):
 if n == 0:
 return 1
 else:
 return p * power (p, n - 1)
 p = float (input ("Enter principal/base value: "))
n = int (input ("Enter number of years/powers: "))
 result = power (p, n)
print("Power =", result)
```

OUTPUT:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab1b.py ==========
Enter principal/base value: 456
Enter number of years/power: 23
Result = 1432819163011061360894656451138504049716965586066841347398041 6
```

EXPLANATION:

- The function of power (p, n) calculates p^n using recursion by multiplying p with the result of power (p, n-1).
- The recursion stops when n becomes 0, returning 1, which is the base case.
- The final result is displayed after all recursive calls are completed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Type Error	Convert the input to float or int using float(input()) or int(input()).
2.	Incorrect output if the recursive call is written as power(p, n + 1) instead of power(p, n - 1).	Change the recursive call to power(p, n - 1) so that n decreases toward the base case.

3.AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search (list, key, index):
 if index >= len(list):
 return False
 if list[index] == key:
 return True
 return search (lst, key, index + 1)
 employees = [101, 102, 103, 104, 105]
 key = int (input ("Enter Employee ID to search: "))
```

```
if search (employees, key, 0):

print (f"Employee ID found at position {key}")

else:

print ("Employee ID Not Found")
```

OUTPUT:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab1c.py =========
Enter employee ID to search: 105
Employee ID found at position 5
```

EXPLANATION:

- The recursive function checks each employee ID in the list one by one, starting from the first element.
- If the required ID is found, it returns True; otherwise, it continues searching recursively until the end of the list.
- If the entire list is searched without finding the ID, the function returns False.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Name Error if the list name is misspelled (e.g., employee instead of employees).	Use the correct variable name consistently throughout the program.
2.	Syntax Error due to a missing colon (:) after an if statement or function definition.	Add the missing colon (:) after if, else, or def.

4.AIM: In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
 if n == 0 or n == 1:
 return 1
 else:
 return n * factorial (n - 1)
num = int (input ("Enter a number: "))
if num < 0:
 print ("Factorial does not exist for negative numbers.")
else:
 print ("Factorial =", factorial(num))
```

OUTPUT:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab1d.py =======
Enter a number: 14
Factorial = 87178291200
```

EXPLANATION:

- The function factorial(n) computes the factorial by multiplying the current number with the factorial of the previous number.
- The recursion ends when n becomes 0 or 1, where the factorial value is 1.
- The program also checks for negative inputs since factorial is defined only for non-negative integers.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError if the statements inside the function are not properly indented.	Maintain proper indentation (usually 4 spaces) inside the function.
2.	TypeError if a decimal number (e.g., 5.5) is entered for factorial calculation.	Enter only a whole number or convert the input using int().

5.AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fibonacci(n):
 if n == 0:
 return 0
 elif n == 1:
 return 1
 else:
 return fibonacci(n - 1) + fibonacci(n - 2)
n = int (input ("Enter the number of terms: "))
print ("Fibonacci Series:")
for i in range(n):
 print (fibonacci(i), end=" ")
```

OUTPUT:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lable.py ==========
Enter number of terms: 32
Fibonacci series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711
 28657 46368 75025 121393 196418 317811 514229 832040 1346269
```

EXPLANATION:

- The function Fibonacci(n) returns the nth Fibonacci number by recursively adding the previous two Fibonacci numbers.
- The base cases return 0 for the first term and 1 for the second term.
- A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both base conditions before the recursive call.
2.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.

LAB-2

1.AIM: Write a program to perform linear search on a given input.

PROGRAM:

```
def linear_search(arr,key):
 for i in range(len(arr)):
 if arr[i]==key:
 return i
 return -1
 n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
 arr.append(int(input()))
key=int(input("Enter element to search:"))
result=linear_search(arr,key)
if result!=-1:
 print("Element found at index",result)
else:
 print("Element not found.")
```

OUTPUT:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab2a.py ==========
Enter elements: 43 54 65 34 65 321 76 3
Enter element to search: 321
Element found at position 6
```

EXPLANATION:

The linear_search() function checks each element of the list one by one and compares it with the given key.

- If the key is found, its index is returned; otherwise, the function returns -1.
- The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2.AIM: Write a Python program to perform binary search on the given input. Verify by taking sorted array and unsorted array as input.

PROGRAM:

```
def binary_search(arr,key):

 low=0

 high=len(arr)-1
```

```
while low<=high:
 mid=(low+high)//2
 if arr[mid]==key:
 return mid
 elif arr[mid]<key:
 low=mid+1
 else:
 high=mid-1
 return -1


n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
 arr.append(int(input()))


key=int(input("Enter element to search:"))
result=binary_search(arr,key)
if result!=-1:
 print("Element found at index",result)
else:
 print("Element not found.")
```

OUTPUT:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab2a.py ==========
Enter elements: 43 54 65 34 65 321 76 3
Enter element to search: 321
Element found at position 6
```

PROGRAM:

```
def binary_search(arr,key):
 low=0
 high=len(arr)-1
 while low<=high:
 mid=(low+high)//2
 if arr[mid]==key:
 return mid
 elif arr[mid]<key:
 low=mid+1
 else:
 high=mid-1
 return -1
 n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
 arr.append(int(input()))
 if arr==sorted(arr):
 print("\n The input list is already sorted.")
else:
```

```
 print("\n The input list is not sorted.")

 print("Sorting the list...")

 arr.sort()

print("Sorted list.",arr)

 key=int(input("Enter element to search:"))

result=binary_search(arr,key)

if result!=-1:

 print("Element found at index",result)

else:

 print("Element not found.")
```

OUTPUT:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab2b2.py =========
Enter elements: 542 43 53 756 423 67 342
Sorted array: [43, 53, 67, 342, 423, 542, 756]
Enter element to search: 43
Element found at position 1
```

EXPLANATION:

- The binary_search() function repeatedly divides the sorted list into two halves and compares the middle element with the search key.
- If the middle element is smaller than the key, the search continues in the right half; otherwise, it searches the left half.
- The second program verifies whether the input is sorted and sorts it using arr.sort() if necessary before performing binary search.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly.	Initialize low = 0, high = len(arr) - 1 and update them as low = mid +1 or high = mid - 1.

Lab-3

1.AIM: write programs for the Bubble sort

PROGRAM:

```
#bubble sort
def bubble_sort(arr):
    n = len(arr)
    for i in range (n-1):
        for j in range(n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1] = arr[j+1],arr[j]
    return arr
#taking input
n = int(input("Enter number of elements: "))
arr=[]
print("Enter elements: ")
for i in range(n):
    arr.append(int(input()))
bubble_sort(arr)
print("sorted array: ")
for element in arr:
    print(element, end=" ")
```

Output:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab3a.py ==========
Enter number of elements: 5
Enter elements:
45
3
77
56
54
sorted array:
3 45 54 56 77
```

Explanation:

Bubble sort compares two adjacent elements. If the left element is greater than the right element, they are swapped. After each pass, the largest remaining element moves to the end.

Error table:

S.no	Error	Correction
1.	Used wrong loop range	Use for j in range(n-i-1):
2.	Swap condition is wrong	Use if arr[j] > arr[j+1]:

2.AIM: write programs for the Insertion sort

PROGRAM:

```
# Insertion Sort

 def insertion_sort(arr):

    n = len(arr)

    for i in range(1, n):

        key = arr[i]

        j = i - 1

         while j >= 0 and arr[j] > key:

            arr[j + 1] = arr[j]

            j = j - 1
```

```
        arr[j + 1] = key
    return arr
# Taking input
n = int(input("Enter number of elements: "))
arr = []
print("Enter elements: ")
for i in range(n):
    arr.append(int(input()))
insertion_sort(arr)
print("Sorted array: ")
for element in arr:
    print(element, end=" ")   same for this
```

Output:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab3b.py =====
Enter number of elements: 7
Enter elements:
65
234
76
5
56
45
76
Sorted array:
5 45 56 65 76 76 234
```

Explanation:

Insertion sort builds the sorted array one element at a time.

It takes one element called the **key**, compares it with the elements before it, shifts larger elements to the right, and inserts the key in the correct position.

Error table:

S.no	Error	Correction
1.	key is not stored properly	Use key = arr[i]
2.	arr[j+1] = key placed inside while loop	Place it after the while loop

3.AIM: write programs for the Bubble sort

PROGRAM:

```
# Selection Sort
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
# Taking input
n = int(input("Enter number of elements: "))
arr = []
print("Enter elements: ")
for i in range(n):
```

```
    arr.append(int(input()))

selection_sort(arr)

print("Sorted array: ")

for element in arr:

    print(element, end=" ")
```

Explanation:

Selection sort repeatedly finds the **smallest element** from the unsorted part of the array and places it in its correct position.

Error table:

S.no	Error	Correction
1.	min_index is not initialized	Use min_index = i
2.	Swap is placed inside inner loop	Place swap after the inner loop